Programming Assignment 1: Password Cracking
CMPS 315

Objectives:

- Implement a dictionary attack to crack salted, hashed MD5 passwords

Learning Outcomes:

(2) Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline

Description:

Using **Python 3**, write a program to perform a dictionary attack on a file of password hashes that have been salted and encoded with the **MD5** hashing algorithm. The goal is to recover the original passwords. Name your source code **pa1.py**.

Program inputs:

Your program will process two input files. The filenames can be hardcoded into your source code:

1. **Shadow File** (**shadow.txt**): The shadow file containing the hashes to crack

   This file has a format similar to the Linux shadow file with 8 fields separated by colons (**:**) . Since the assignment focuses on the username and password information, the shadow file has blanks for the latter fields:

   ```
   Username:Password:::::::
   ```

   The username can be of **varying length**.

   The **`Password`** field has subfields separated by a **$**:

   ```
   $HashAlg$Salt$HashDigest
   ```

   The salt is unique for all users and contains **5 lowercase letters**. The **`$1`** identifier indicates the use of the **MD5** hashing algorithm. Use MD5 from the Python **hashlib** library. It differs from md5_crypt used in a Linux shadow file. The hash digest is calculated by applying MD5 to (salt+plaintext password).

2. **Dictionary file** (**rockyou.txt**):

   The second file is the dictionary of plaintext passwords to use for the attack and is named **rockyou.txt**. To reduce the computation time, the dictionary contains a subset of compromised passwords from the rockyou wordlist. The sample dictionary on Moodle contains approximately 200,000 passwords. Your program should be able to process a dictionary of **any length**.

Processing:

To crack a password in a dictionary attack, read a word from the dictionary, prepend the salt, and hash it using MD5. If the calculated hash matches the user's stored hash, you have found the password. The dictionary word, however, must exactly match the user's password. Oftentimes users will try to use "clever" variations of these passwords (e.g., capitalizing only the first character). Additionally, users often incorporate their username into the password. When cracking the hashes, you must consider not only the original passwords from the rockyou list, but also the following variations:

Variation	Dictionary Password	Result
All lowercase letters	PASSWORD PasswordD123	password password123
All uppercase letters	password paSSworD123	PASSWORD PASSWORD123
Capitalize first character only (all other characters are lowercase)	password123 PASSWORD 123abc	Password123 Password 123abc
Capitalizing last character only (all other characters are lowercase)	Password123 Password	password123 passworD
Rotate the password right by one charcter (i.e., the last character becomes the first)	chocolate	echocolat
Mirror (append reversed string)	Password abc123	PassworddrowssaP abc123321cba
Replace letter o/O (upper and lowercase) with the number `0`.	chocolate PASSWORD	ch0c0late PASSW0R D
Append username to the end of password	Username: Bob password123 chocolate	 password123Bob chocolateBob

Program outputs:

Output is displayed to the **console** (not written to a file).

- For each cracked password display: **username, salt, hash, and cracked plaintext password**.
- If a password is **not found**, display: **username, salt, and the uncracked hash**.
- Also display a summary count of the **number of matched hashes** (passwords cracked) and a count of the **number of unmatched hashes** (passwords not cracked).

Testing:

Your program will be tested using Python3 with a unique (unseen) shadow file.

Coding Guidelines and Restrictions:

This assignment can be completed using only the Python concepts covered in CMPS 150 and the provided tutorial on Moodle. Your solution should not include dictionaries, enumerations, classes and object-oriented programming, or any external libraries (except hashlib for MD5). You should not use unfamiliar and overly advanced syntax.

Your code should reflect your own understanding and problem-solving skills. You may not copy code from online sources or use AI-generated code (e.g., ChatGPT, GitHub Copilot). Any submissions suspected of plagiarism or unauthorized assistance will be subject to academic integrity policies.

Additional Requirements:

- You must comment your code appropriately to explain its logic.
- Other than **hashlib** for MD5, you may not import external libraries.
- Do not use prewritten password-cracking tools. You must write your own implementation.
- Include the following header in your source code

```
# Name:        Type-Your-Name
# ULID:        Type-Your-ULID
# Course:      CMPS 315
# Assignment: pa1 - Password racking
# Certification of Authenticity:
# I certify that this assignment is entirely my own work.
```

What to submit:

- Submit your Python source code file named **pa1.py**.

Sample Run:

```
Sample rockyou.txt Dictionary

superman
chocolate
!@#$%
123456789
Grizzly
password18
cheyenne
a123456
j4**9c+p
fluffy1
123four
```

```
Passwords used to create shadow.txt:

bobbyS superman
sarahK abc123
fritzP grizzly
brandL fluffy11yffulf
clydeM Cheyenne
dirksY echeyenn
faintR chocolatE
zonesA ch0c0late
grapeS J4**9C+P
brownE password
flintB 123fourflintB
```

```
Sample: shadow.txt

bobbyS:$1$ycszt$dded4c8d2f0433efdd19f2c78d308aef:::::::
sarahK:$1$dxhwm$baf7b7549032dd71e098a05b11ae660d:::::::
fritzP:$1$ogkib$8d0355f325bbf8787a36bb984346d3e7:::::::
brandL:$1$imfut$76927b1b99008dc789dde11f017bac27:::::::
clydeM:$1$iifir$edb9e5909f6711cd6162d806f89e916d:::::::
dirksY:$1$vsbop$30ba17f1e1e26a88f8828d224777f4ee:::::::
faintR:$1$wttnm$663725c3d477da54ed3c977b3cbaedcd:::::::
zonesA:$1$lcswn$2543045050d9a23fb037331d9e08137b:::::::
grapeS:$1$kebtq$3dc2ef1a835d51fc6618830941f1e280:::::::
brownE:$1$bqune$9e207da138a100ef3ece8172f36b9759:::::::
flintB:$1$ysgol$54c3f7357ce69a94f14173a1a38543e1:::::::
```

Sample output for the sample **shadow.txt** file:

Name	Salt	Hash	Password
bobbyS	ycszt	dded4c8d2f0433efdd19f2c78d308aef	superman
sarahK	dxhwm	baf7b7549032dd71e098a05b11ae660d	
fritzP	ogkib	8d0355f325bbf8787a36bb984346d3e7	grizzly
brandL	imfut	76927b1b99008dc789dde11f017bac27	fluffy11yffulf
clydeM	iifir	edb9e5909f6711cd6162d806f89e916d	Cheyenne
dirksY	vsbop	30ba17f1e1e26a88f8828d224777f4ee	echeyenn
faintR	wttnm	663725c3d477da54ed3c977b3cbaedcd	chocolatE
zonesA	lcswn	2543045050d9a23fb037331d9e08137b	ch0c0late
grapeS	kebtq	3dc2ef1a835d51fc6618830941f1e280	J4**9C+P
brownE	bqune	9e207da138a100ef3ece8172f36b9759	
flintB	ysgol	54c3f7357ce69a94f14173a1a38543e1	123fourflintB

```
Cracked Hashes:    9
Uncracked Hashes: 2
```

Background knowledge:

- The RockYou wordlist

  RockYou was a company that experienced a data breach in 2009 which exposed over 14 million unique passwords.. The company stored plaintext passwords. The full list of exposed passwords is readily available and frequently used in dictionary attacks. RockYou didn't enforce any password complexity requirements, and so unsurprisingly most of the passwords are very basic. Therefore, many of the passwords are not particularly useful in their original form for cracking password hashes belonging to stronger systems.

- MD5 hash in Python:

  Python's **`hashlib`** contains several different hash and message digest algorithms including MD5.

  Python strings use the Unicode standard, which represents characters by code points. The methods/functions in **`hashlib`** operate on **byte data**, so all strings must be converted to a byte format prior to hashing (translating the string into a sequence of bytes is called encoding).

  If the input to the hash function is a string **variable** - use **`encode`** to convert it to bytes.

```
# stringVariable --> a variable containing string data
import hashlib
hashObject = hashlib.md5( stringVariable.encode('utf-8') )
hashHex = hashObject.hexdigest()
print (hashHex)
```

  Using **`stringVariable.encode()`** returns a bytes representation of the string variable, `stringVariable`. The UTF-8 parameter indicates the encoding scheme used; UTF-8 is the most common and used by default. You can also write:

```
hashObject = hashlib.md5(stringVariable.encode())
```

  The **`md5()`** method returns an md5 **object**. To display the hex value of the secure hash, use the **`hexdigest()`** method (the output of this method is a string).